Assignment-02

* Aim: Write a C program for sparce matrix initialization & operations on it
  → Transpose → Fast Transpose

* Objective:
  → To study the concept of sparce matrix, how it is stored & displayed.

  → To understand the concepts of implementation of sparce matrix operations.

* Platform:
  → 64-bit open source linux or its derivative.
  → Open source C programming tool.

* Pseudo Code:

1. Input matrix to compact form

START

Read rows, cols
K ← 1
for i=0 to rows-1
  for j=0 to col-1
    Read matrix[i][j]

```
            If  matrix[i][j] ≠0

                compact[k].row = i
                compact[k].col = j
                compact[k].value = matrix[i][j]

                k= k+1
        End If
    End for
End for

compact[0].row =  rows
compact[0].col=  cols
compact[0].value=  k-1

STOP
```

2. Simple Transpose

```
Transpose[0].row = original.col
Transpose[0].col = original.rows
Transpose[0].value= original.non-zero

k ← 1

for each  column c
          for each non-zero element
```

```
        If element.coloumn = C
            Transpose[k].row = element.coloumn
            Transpose[k].col = element.row
            Transpose[k].value = element.value

            k = k+1
        End If
    End for
End for

Stop
```

3. Fast Transpose

```
Start

Store transpose head

initialize Total[] = 0

For each non-zero element
        Total[coloumn]++

End for

INDEX[0] = 1
```

```
for i = 1 to cols - 1
        Index [i] = Index [i-1] + Total [i-1]
End for
for each non zero element

        position = INDEX [element. column]

        fast [position]. row  = element . column
        fast[position]. col = element. row
        fast[position]. value = element. value

        Index [element. column] ++

End for

STOP
```

* Time Complexity

Operation	Time complexity
• Compact Representation	O(mxn)
• Simple Transpose	O(cols x non-zero elements
• Fast Transpose	O(cols + non-zero element

- Conclusion:

Thus, implemented sparce matrix operation. This system is able to perform diff. operations on sparce matrix such as simple & fast transpose & their time complexity.

* FAQs:

Q1) What is a sparce matrix? List the applications.

A- Sparce Matrix:

It has most of the element as zero[0] & only a few elements are non-zero.

Ex:

0	0	3	0	4
0	0	5	7	0
0	0	0	0	0
0	2	6	0	0

- Application:

→ Graph Representation
→ Image Processing
→ Computer Graphics
→ Network Analysis
→ Database System.

2) Represent Sparce matrix with suitable data structure. Explain simple & fast transpose with an example.

A- Representation of SM:

- Triplet (compact) Rep ← most common.

each non zero member is stored as

(Row, Column, Value)

First Row ↓
(no. of rows, no. of cols, no. of non-zero element)

Ex:

0	0	3	0	4
0	0	5	7	0
0	0	0	0	0
0	6	2	0	0

Triplet Representation

Row	Cols	Value
4	5	6
0	2	3
0	4	4
1	2	3
1	3	7
3	1	2
3	2	6

Explaination :
1st row
- 4 row
- 5 colums
- 6 non-zero elem

Remaining rows contains

Simple Transpose :

Converts rows into columns by scanning one-by-one columns.

Algo: 1. Copy header row
2. For each column:
→ Store all non zero elements
→ When column matches, interchange rows & columns.

Ex:

Original

Row	Col	Value
4	5	6
0	2	3
0	4	4
1	2	5
1	3	7
3	1	2
3	2	6

ST

Row	Col	Value
5	4	6
1	3	2
2	0	3
2	1	5
2	3	6
3	1	7
4	0	4

Time Complexity :

O(Columns x Non zero elements)

Fast Transpose : Improves efficiency by using more arrays

- Total[] - store non-zero element
- Index[] - stores slashing position.

- Algorithm:

1. Copy the header
2. Count non-zero element using Total()
3. Compute starting position using Index()
4. Place every element directly into its convert position.
5. Increment corresponding Index.

Total[]

Column	0	1	2	3	4
Count	0	1	3	1	1

Index[]

Colum	0	1	2	3	4
Startpos	1	1	2	5	6

Final:

Row	Col	Value
5	4	6
1	3	2
2	0	3
2	1	5
2	3	6
3	1	7
4	0	1

Time Complexity:

O(column + non zero element)

- Find out addition of 2 sparce matrix in triplet form & also find simple & fast transpos.

M1:

4	5	6
0	3	5
1	3	6
1	4	45
2	3	4
3	2	45
4	1	2

M2:

4	5	6
0	3	7
0	4	6
1	4	4
2	1	8
3	2	45
4	4	21

Addition:

Pos	M_1	M_2	Sum
(0,3)	5	7	12
(0,4)	0	6	6
(1,3)	8	0	8
(1,4)	45	4	49
(2,1)	0	8	8
(2,3)	4	0	4
(3,2)	45	45	90
(4,1)	2	0	2

Total Non-zero element = 9

- Result in Triplet form

Row	col	Value
4	5	9
0	3	12
0	4	6
1	3	8
1	4	49
2	1	8
2	3	4
3	2	90
4	1	2
4	4	2

- Simple Transpose:

Row	Col	Value
5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	45

- Fast Transpose:

Row	Col	Value
5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	45

total()

Column

Count	0	1	2	3	4
	0	1	1	3	1

JV
23/7/20

Code:

```
#include <stdio.h>
#define MAX 100
void compact(int A[MAX][MAX], int m, int n, int B[MAX][3]) {
   int k = 1;
   B[0][0] = m;
   B[0][1] = n;

   for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++) {
         if (A[i][j] != 0) {
            B[k][0] = i;
            B[k][1] = j;
            B[k][2] = A[i][j];
            k++;
         }
      }
   }
   B[0][2] = k - 1;
}

void simpleTranspose(int A[MAX][3], int B[MAX][3]) {
   int m = A[0][0];
   int n = A[0][1];
   int t = A[0][2];

   B[0][0] = n;
   B[0][1] = m;
   B[0][2] = t;

   if (t <= 0) return;

   int q = 1;
   for (int col = 0; col < n; col++) {
      for (int p = 1; p <= t; p++) {
         if (A[p][1] == col) {
            B[q][0] = A[p][1];
            B[q][1] = A[p][0];
            B[q][2] = A[p][2];
            q++;
         }
      }
   }
}
void fastTranspose(int A[MAX][3], int B[MAX][3]) {
   int m = A[0][0];
   int n = A[0][1];
```

```
    int t = A[0][2];

    int S[MAX], T[MAX];

    B[0][0] = n;
    B[0][1] = m;
    B[0][2] = t;

    if (t <= 0) return;

    for (int i = 0; i < n; i++) {
        S[i] = 0;
    }

    for (int i = 1; i <= t; i++) {
        S[A[i][1]] = S[A[i][1]] + 1;
    }

    T[0] = 1;
    for (int i = 1; i < n; i++) {
        T[i] = T[i - 1] + S[i - 1];
    }

    for (int i = 1; i <= t; i++) {
        int j = A[i][1];
        int pos = T[j];

        B[pos][0] = A[i][1];
        B[pos][1] = A[i][0];
        B[pos][2] = A[i][2];

        T[j]++;
    }
}

void displaySparse(int B[MAX][3]) {
    int t = B[0][2];
    printf("\nRow\tCol\tValue\n");
    printf("-----------------------\n");
    for (int i = 0; i <= t; i++) {
        printf("%d\t%d\t%d\n", B[i][0], B[i][1], B[i][2]);
    }
}

int main() {
    int A[MAX][MAX], C[MAX][3], B[MAX][3];
    int m, n, ch;
```

```
int compactCreated = 0;

do {
   printf("\n--- MENU ---");
   printf("\n1. Compact Form (Input Matrix)");
   printf("\n2. Simple Transpose");
   printf("\n3. Fast Transpose");
   printf("\n4. Exit");
   printf("\nEnter your choice: ");
   scanf("%d", &ch);

   switch (ch) {
      case 1:
         printf("Enter nos of rows (m) and columns (n): ");
         scanf("%d %d", &m, &n);

         printf("Enter matrix elements of A:\n");
         for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
               scanf("%d", &A[i][j]);
            }
         }

         compact(A, m, n, C);
         compactCreated = 1;
         printf("\nCompact form created in matrix C:");
         displaySparse(C);
         break;

      case 2:
         if (!compactCreated) {
            printf("\nPlease input matrix (Option 1) first!\n");
         } else {
            simpleTranspose(C, B);
            printf("\nSimple Transpose complete in matrix B:");
            displaySparse(B);
         }
         break;

      case 3:
         if (!compactCreated) {
            printf("\nPlease input matrix (Option 1) first!\n");
         } else {
            fastTranspose(C, B);
            printf("\nFast Transpose complete in matrix B:");
            displaySparse(B);
         }
```

```
            break;

        case 4:
            printf("\nExiting program.\n");
            break;

        default:
            printf("\nInvalid choice!\n");
      }
   } while (ch != 4);

   return 0;
}
```

Input and Output:

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:1

Enter nos of rows (m) and columns (n):4 4

Enter matrix elements of A:
1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1


Compact form created in matrix C:
Row     Col     Value
-----------------------
4       4       4
0       0       1
1       1       1
2       2       1
3       3       1
```

```
Enter your choice:2


Simple Transpose complete in matrix B:
Row     Col     Value
-----------------------
4       4       4
0       0       1
1       1       1
2       2       1
3       3       1

--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:3


Fast Transpose complete in matrix B:
Row     Col     Value
-----------------------
4       4       4
0       0       1
1       1       1
2       2       1
3       3       1
```

```
--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:5


Invalid choice!

--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:4


Exiting program.

Process finished with exit code 0
```